



琴音袅袅

—— 一个乐器交易租借平台 ——

详细设计规约

指导教师：杜庆峰

小组成员

陈甫彬 2253715

刘子赫 2250694

杨兆镇 2252712

修订历史

编写日期	SEPG	版本	说明	作者
2024.12.01		1.0	根据概要设计，完成详细设计内容框架内容	陈甫彬、刘子赫、杨兆镇
2024.12.03		1.1	对详细设计内容的接口进行详细设计	陈甫彬、刘子赫、杨兆镇
2024.12.07		1.2	再一次改善了类接口调用的规范	陈甫彬、刘子赫、杨兆镇
2024.12.10		1.3	进行了细微修改，最终确定了详细设计规约	陈甫彬、刘子赫、杨兆镇

目录

1 引言	1
1.1 编写目的	1
1.2 背景及依据	1
1.3 参考资料	1
2 系统软件结构	2
3 微服务设计	3
4 类设计	4
4.1 登录注册服务	4
4.2 AI 对话服务	10
4.3 收藏商品服务	11
4.4 评论服务	14
4.5 支付订单管理服务	18
4.6 卖出订单管理服务	21
4.7 支付服务	23
4.8 商品管理服务	27

1. 引言

1.1 编写目的

本文档是基于上一次的概要设计规约，进行进一步细化的接口设计的详细设计规约文档。我们将从微服务设计、接口设计、类详细设计三个角度，将概要设计中不够细化的接口信息，接口属性，方法签名等进一步详细化，服务于后续实际的编码过程。

1.2 背景及依据

本项目名为琴音袅袅，是一款发布在 Web 端上的乐器交易平台应用。旨在让用户可以在应用中进行乐器相关商品的交易、租借活动。基于最开始的需求诱导下的需求调研，我们得出了用户对乐器租借的欲望非常强烈，所以我们的平台需要加入租借功能。做一个购买和租借两集合的交易平台。确保想学习乐器但又担心高价买入放弃的用户可以尝试乐器的学习过程。同时，近期的生成式 AI 大火，解决了许多人的一系列生活方面的问题，在此背景下，我们也希望给该应用引入一个生成式 AI 接口，让它可以指导用户进行乐器的选择、乐器的学习过程。

本项目任务的提出者为软件工程课程设计需要；开发者为同级大学软件工程专业三名同学（陈甫彬、刘子赫、杨兆镇）。

项目预期用户为：音乐爱好者、想尝试学习乐器又担心高价买入而放弃浪费的用户等。

为了更好的实现我们的功能，本系统预计调用（微信、支付宝）支付接口、手机号验证接口以及生成式 AI 接口。

1.3 参考资料

• 概要设计规约

标题：乐器交易平台概要设计规约

文件编号：20241125-NA-001

发表日期：2024 年 11 月 25 日

出版单位：课程项目组

来源：小组内部文档

• 用户调研问卷

标题：乐器交易平台用户调研与问卷分析

文件编号：20241020-UR-001

发表日期：2024 年 10 月 20 日

出版单位：软件工程课程小组

来源：小组内部文档

• 系统设计标准

标题：Web 应用系统设计与开发标准

文件编号：WASDS-2023

发表日期：2023 年 6 月 1 日

出版单位：国际软件工程协会（ISEA）

来源：<https://www.isea.org/>

• 技术文档

标题：Vue.js 官方文档

文件编号：VJS-2023

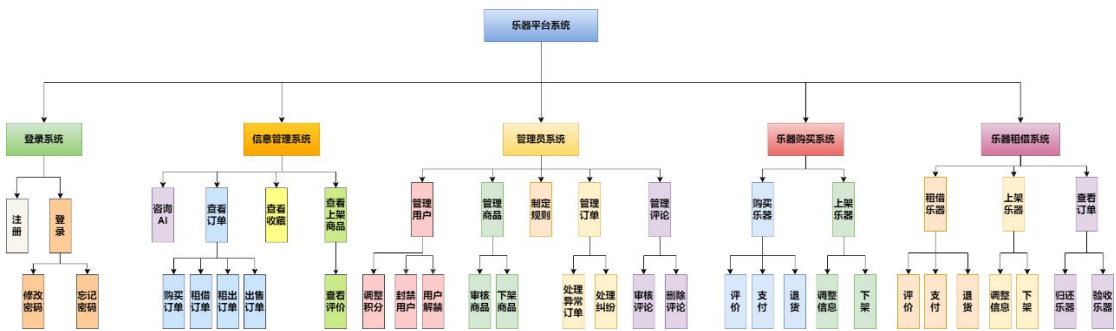
发表日期：2023 年 3 月 10 日

出版单位：Vue.js 官方团队

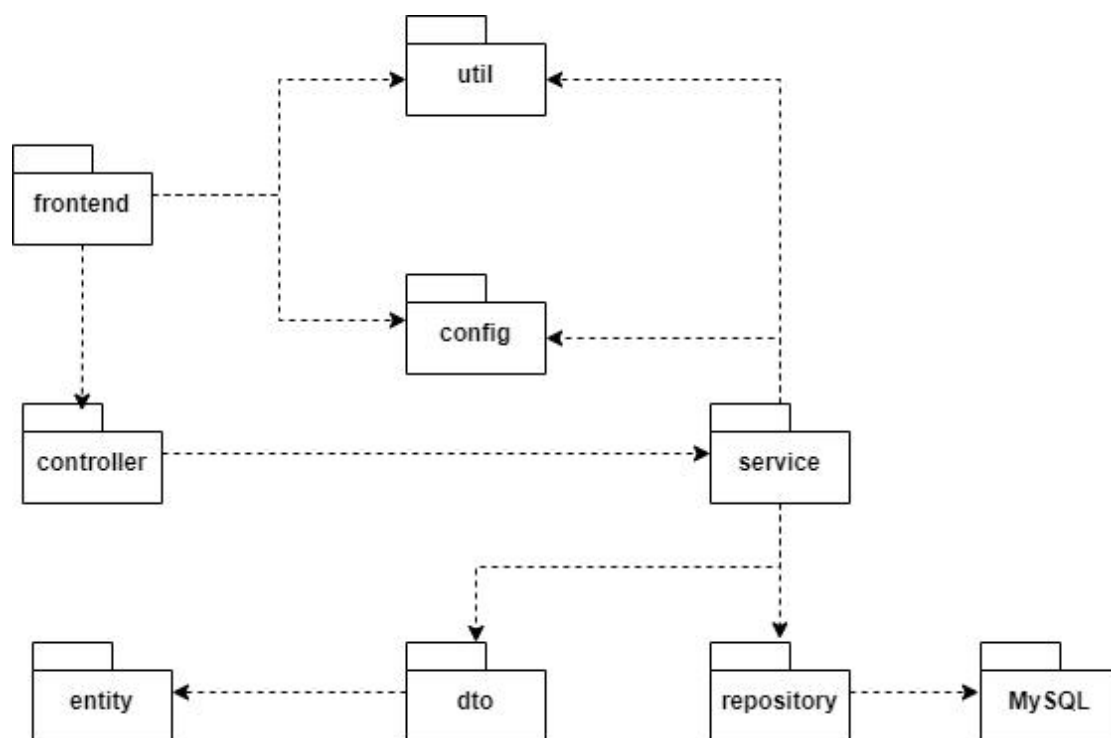
来源：<https://vuejs.org/>

2. 系统软件结构

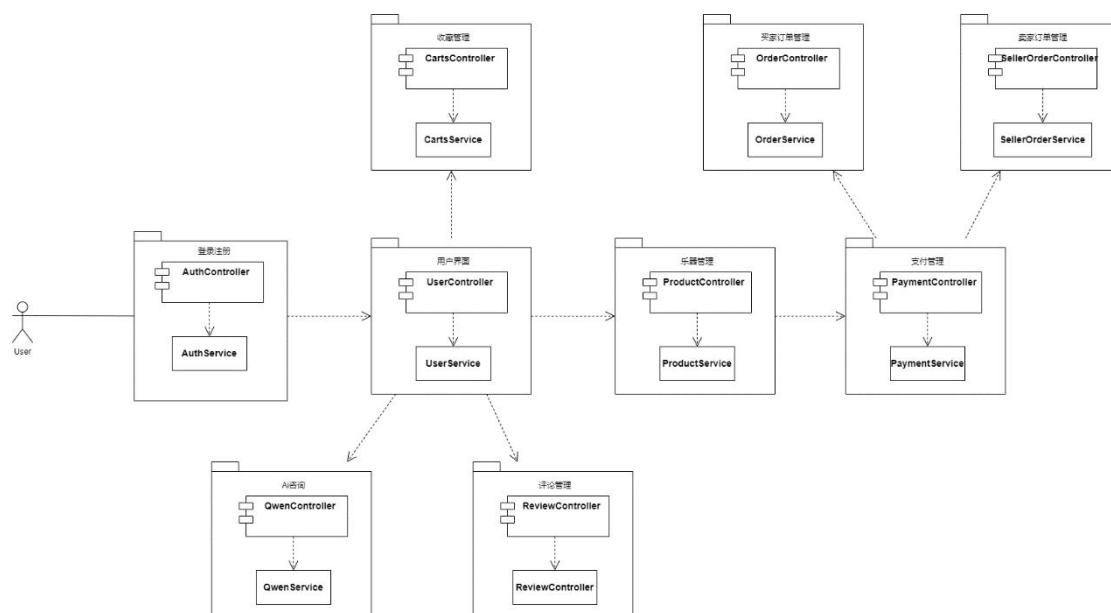
• 子系统结构图



• 体系结构包图

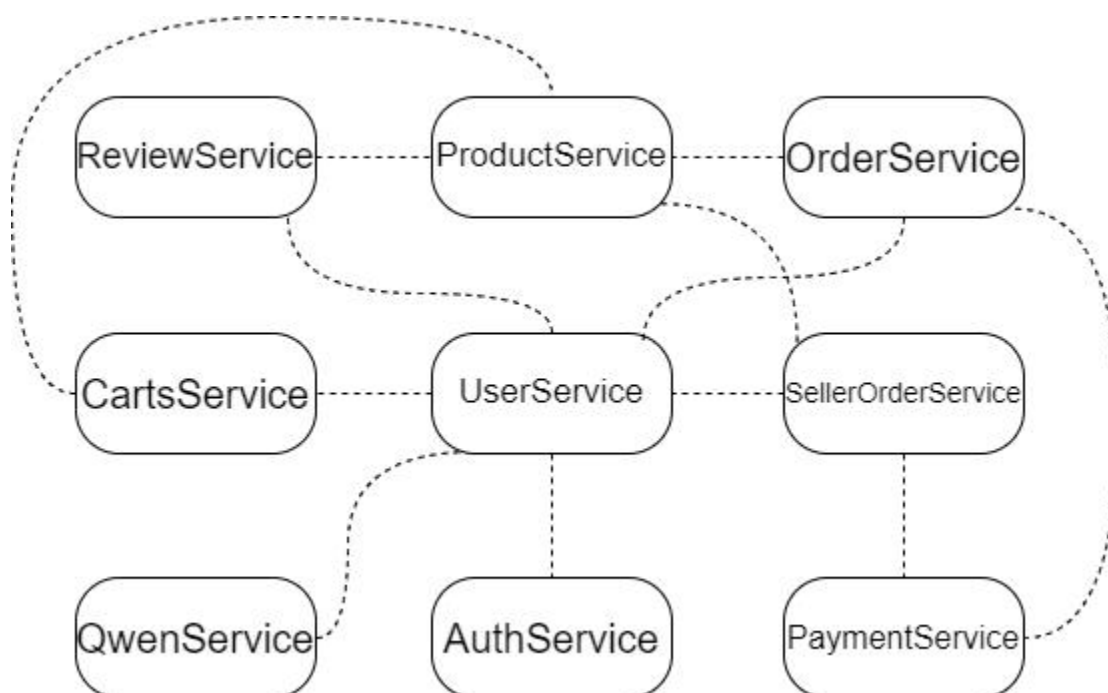


• 软件架构设计



基于 java Spring boot

3. 微服务设计



基于以上的服务架构图，我们设置有：

UserService，用于用户的信息管理服务。

AuthService, 短信认证服务。

QwenService, AI 交互千问服务。

PaymentService, 支付服务

SellerOrderService, 出售订单管理服务。

OrderService, 购买订单管理服务。

ProductService, 商品管理服务。

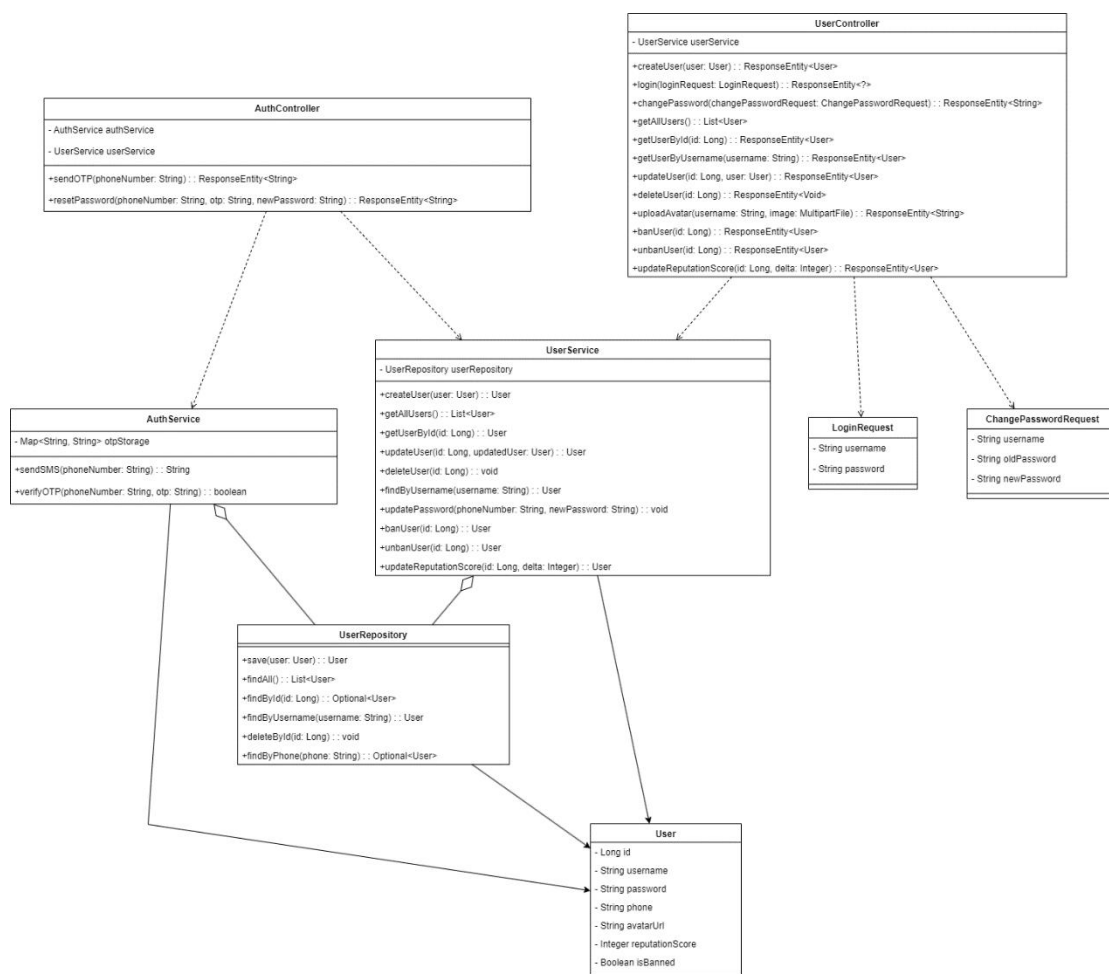
ReviewService, 评论管理服务。

CartsService, 购物车管理服务。

4. 类设计

4.1 登录注册服务

有关键的实体类 User, 代表每一个用户的信息。基于 Java Spring Boot 结构, Service 处理业务逻辑; Controller 为接口类, 提供方法; Repository 为持久类, 用于访问操作数据库。这里的 Auth 业务类用于短信认证服务。



4.1.1 AuthController 类

4.1.1.1 功能

提供与用户认证相关的接口，包括发送验证码和重置密码。

使用 RESTful 风格设计，通过 HTTP 请求实现用户的认证功能。

4.1.1.2 性能

sendOTP 方法通过调用 AuthService 的短信发送服务，确保快速响应用户请求。

resetPassword 方法校验验证码并更新用户密码，提供高效的认证服务。

接口采用轻量化的 HTTP POST 方法，适合高并发环境。

4.1.1.3 输入项

手机号 (phoneNumber)：用于标识用户。

验证码 (otp)：用于验证用户身份。

新密码 (newPassword)：用于设置新的登录密码。

4.1.1.4 输出项

成功或失败的操作结果信息，返回 HTTP 状态码和相关消息内容。

4.1.1.5 方法详细说明

4.1.1.5.1 sendOTP 方法

- 方法签名

`ResponseEntity<String> sendOTP(String phoneNumber)`

- 功能说明

发送验证码到用户指定的手机号，用于验证用户身份。

- 接口调用

调用 `AuthService` 的 `sendSMS(phoneNumber)` 方法。

- 参数列表

`phoneNumber`: `String` 类型，用户手机号。

- 返回列表

`ResponseEntity<String>`: 验证码发送结果信息。

4.1.1.5.2 resetPassword 方法

- 方法签名

`ResponseEntity<String> resetPassword(String phoneNumber, String otp, String newPassword)`

- 功能说明

校验用户输入的验证码，若校验成功则更新用户密码

- 接口调用

调用 `AuthService` 的 `verifyOTP(phoneNumber, otp)` 方法验证验证码。

调用 `UserService` 的 `updatePassword(phoneNumber, newPassword)` 方法更新密码。

- 参数列表

`phoneNumber`: `String` 类型，用户手机号。

`otp`: `String` 类型，用户输入的验证码。

`newPassword`: `String` 类型，用户输入的新密码。

- 返回列表

`ResponseEntity<String>`: 密码重置结果信息。

4.1.1.6 限制条件及出错处理

用户手机验证不对进行限制。

4.1.2 UserController 类

4.1.2.1 功能

提供用户管理相关的 RESTful 接口，包括用户的注册、登录、修改密码、查询用户信息、更新用户信息、上传头像、封禁用户、调整信誉积分等功能。

4.1.2.2 性能

通过 Spring Boot 的 `@RestController` 和 `@RequestMapping` 注解实现高效的 HTTP 请求处理。

使用轻量级 HTTP 方法（GET、POST、PUT、DELETE）适配各类操作。

合理使用 `@Autowired` 注入 `UserService`，提高接口逻辑复用性和易维护性。

4.1.2.3 输入项

用户信息：User 对象，用于注册、更新用户信息等操作。

用户名：String 类型，用于查询或更新用户信息。

用户 ID：Long 类型，用于标识用户。

图片文件：MultipartFile 类型，用于上传用户头像。

调整值：Integer 类型，用于调整信誉积分。

其他请求参数：包括用户名、密码、新密码、验证码等。

4.1.2.4 输出项

成功或失败的操作结果，返回 HTTP 状态码和相关消息内容。

用户实体对象或其部分信息（如用户名、头像路径等）。

4.1.2.5 方法详细说明

4.1.2.5.1 createUser 方法

- 方法签名

```
ResponseEntity<User> createUser(@RequestBody User user)
```

- 功能说明

注册新用户，将用户信息存储到数据库。

- 接口调用

调用 `UserService` 的 `createUser(User user)` 方法。

- 参数列表

user: User 类型，包含用户的注册信息。

- 返回列表

`ResponseEntity<User>`: 返回注册成功的用户信息。

4.1.2.5.2 login 方法

- 方法签名

```
ResponseEntity<?> login(@RequestBody LoginRequest loginRequest)
```

- 功能说明

验证用户的登录信息，包括用户名和密码。

- 接口调用

调用 `UserService` 的 `findByUsername(String username)` 方法。

- 参数列表

`loginRequest`: `LoginRequest` 类型，包含用户名和密码。

- 返回列表

`ResponseEntity<?>`: 返回登录成功的用户信息或错误消息。

4.1.2.5.3 `changePassword` 方法

- 方法签名

```
ResponseEntity<String> changePassword(@RequestBody  
ChangePasswordRequest changePasswordRequest)
```

- 功能说明

验证原密码并更新用户的登录密码。

- 接口调用

调用 `UserService` 的 `findByUsername(String username)` 和 `updateUser(Long id, User user)` 方法。

- 参数列表

`changePasswordRequest`: `ChangePasswordRequest` 类型，包含用户名、原密码和新密码。

- 返回列表

`ResponseEntity<String>`: 返回密码修改结果信息。

4.1.2.5.4 `getAllUsers` 方法

- 方法签名

```
List<User> getAllUsers()
```

- 功能说明

查询所有用户的基本信息。

- 接口调用

调用 `UserService` 的 `getAllUsers()` 方法。

- 参数列表

无。

- 返回列表

`List<User>`: 所有用户的信息列表。

4.1.2.5.5 `uploadAvatar` 方法

- 方法签名

`ResponseEntity<String> uploadAvatar(String username, MultipartFile imageFile)`

- **功能说明**

上传用户头像并更新用户的头像路径。

- **接口调用**

调用 `UserService` 的 `findByUsername(String username)` 和 `updateUser(Long id, User user)` 方法。调用 `FileUploadUtil.saveFile` 和 `FileUploadUtil.deleteFile` 方法处理文件。

- **参数列表**

`username`: `String` 类型，用户的用户名。

`imageFile`: `MultipartFile` 类型，用户上传的头像文件。

- **返回列表**

`ResponseEntity<String>`: 返回上传结果信息。

4.1.2.5.6 banUser 方法

- **方法签名**

`ResponseEntity<User> banUser(Long id)`

- **功能说明**

封禁指定用户。

- **接口调用**

调用 `UserService` 的 `banUser(Long id)` 方法。

- **参数列表**

`id`: `Long` 类型，用户 ID。

- **返回列表**

`ResponseEntity<User>`: 返回封禁后的用户信息。

4.1.2.5.7 updateReputationScore 方法

- **方法签名**

`ResponseEntity<User> updateReputationScore(Long id, Integer delta)`

- **功能说明**

调整用户的信誉积分。

- **接口调用**

调用 `UserService` 的 `updateReputationScore(Long id, Integer delta)` 方法。

- **参数列表**

`id`: `Long` 类型，用户 ID。

`delta`: `Integer` 类型，信誉积分的调整值。

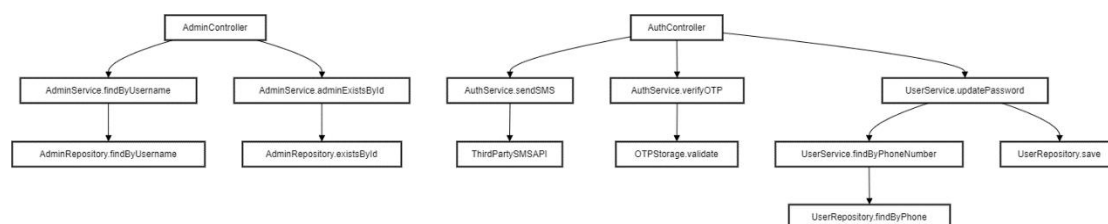
• 返回列表

ResponseEntity<User>: 返回调整后的用户信息。

4.1.2.6 限制条件及出错处理

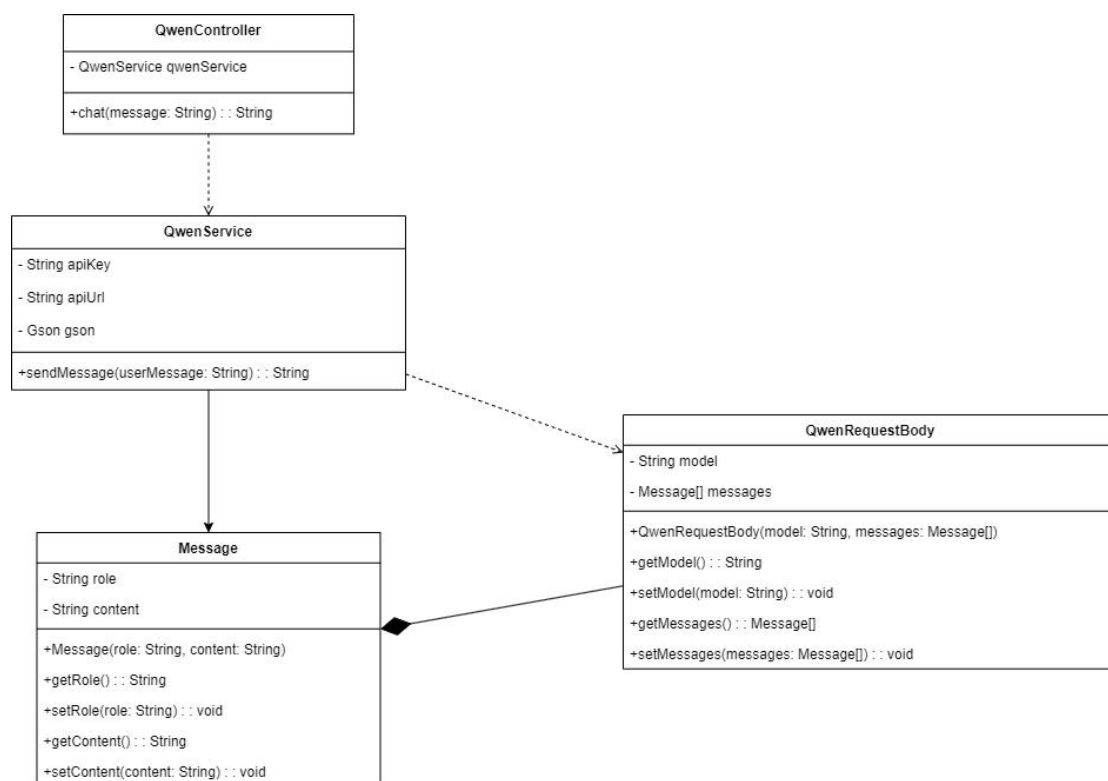
用户输入的密码与用户名不对的时候，进行登录的限制。用户注册不唯一的账号进行限制，用户被封禁进行限制。

4.1.3 类调用图



4.2 AI 对话服务

基于 Java Spring Boot 结构，Service 处理业务逻辑；Controller 为接口类，提供方法；Repository 为持久类，用于访问操作数据库。这里的 Qwen 业务类用于 AI 对话服务。



4.2.1 QwenController 类

4.2.1.1 功能

提供与聊天相关的接口，接收用户发送的消息并调用服务返回回复内容。

4.2.1.2 性能

通过 HTTP POST 请求实现轻量化的消息传递。

使用 QwenService 实现业务逻辑，支持高效处理聊天内容。

4.2.1.3 输入项

用户消息（message）：String 类型，用户发送的文本内容。

4.2.1.4 输出项

聊天回复内容，String 类型，由服务返回的处理结果。

4.2.1.5 方法详细说明

4.2.1.5.1 chat 方法

- 方法签名

String chat(String message)

- 功能说明

接收用户的消息并调用服务处理，返回聊天回复内容。

- 接口调用

调用 QwenService 的 sendMessage(String message) 方法。

- 参数列表

message: String 类型，用户发送的消息。

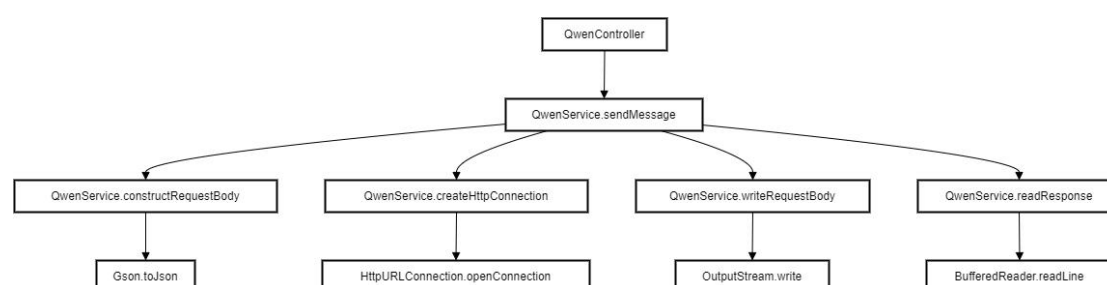
- 返回列表

String: 聊天回复内容。

4.2.1.6 限制条件及出错处理

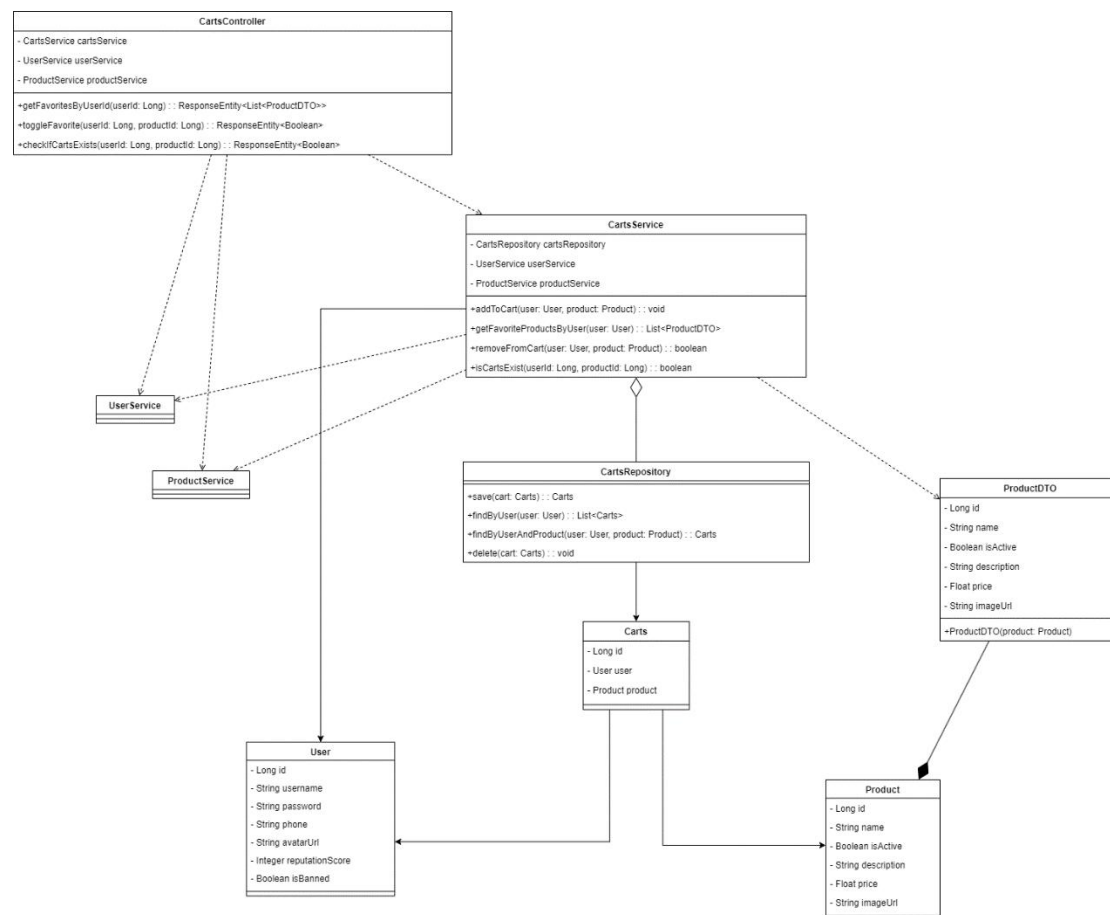
千问提供限制。

4.2.2 类调用图



4.3 收藏商品服务

基于 Java Spring Boot 结构，Service 处理业务逻辑；Controller 为接口类，提供方法；Repository 为持久类，用于访问操作数据库。这里的 Carts 业务类用于购物车收藏服务。



4.3.1 CartController 类

4.3.1.1 功能

提供与购物车（收藏夹）管理相关的接口，包括查询用户收藏的商品、切换收藏状态、检查商品是否在收藏夹中。

4.3.1.2 性能

通过 RESTful 风格的 HTTP 请求提供轻量级接口。

使用 CartService、UserService 和 ProductService 进行分层处理，保证接口逻辑清晰、维护性强。

在 getFavoritesByUserId 方法中过滤未激活的商品，优化返回结果。

4.3.1.3 输入项

用户 ID (userId) : Long 类型，用于标识用户。

商品 ID (productId) : Long 类型，用于标识商品。

4.3.1.4 输出项

查询收藏商品时返回 `List<ProductDTO>` 类型的商品信息列表。

切换收藏状态或检查商品是否存在时返回 `Boolean` 类型的结果。

4.3.1.5 方法详细说明

4.3.1.5.1 chat 方法

- 方法签名

`ResponseEntity<List<ProductDTO>> getFavoritesByUserId(Long userId)`

- 功能说明

根据用户 ID 查询用户收藏的商品列表，仅返回激活状态的商品。

- 接口调用

调用 `UserService` 的 `getUserById(Long userId)` 方法获取用户信息。

调用 `CartsService` 的 `getFavoriteProductsByUser(User user)` 方法获取收藏的商品。

- 参数列表

`userId`: `Long` 类型，用户 ID。

- 返回列表

`ResponseEntity<List<ProductDTO>>`: 用户收藏的激活商品列表。

4.3.1.5.2 toggleFavorite 方法

- 方法签名

`ResponseEntity<Boolean> toggleFavorite(Long userId, Long productId)`

- 功能说明

切换商品在用户收藏夹中的状态（添加或移除）。

- 接口调用

调用 `CartsService` 的 `isCartsExist(Long userId, Long productId)` 方法检查商品是否已在收藏夹中。

根据检查结果调用 `CartsService` 的 `addToCart(User user, Product product)` 或 `removeFromCart(User user, Product product)` 方法。

- 参数列表

`userId`: `Long` 类型，用户 ID。

`productId`: `Long` 类型，商品 ID。

- 返回列表

`ResponseEntity<Boolean>`: 切换后的收藏状态（`true` 表示已收藏，`false` 表示已取消收藏）。

4.3.1.5.3 checkIfCartsExists 方法

- 方法签名

`ResponseEntity<Boolean> checkIfCartsExists(Long userId, Long productId)`

- 功能说明

检查指定商品是否存在于用户的收藏夹中。

- 接口调用

调用 `CartsService` 的 `isCartsExist(Long userId, Long productId)` 方法。

- 参数列表

`userId`: Long 类型，用户 ID。

`productId`: Long 类型，商品 ID。

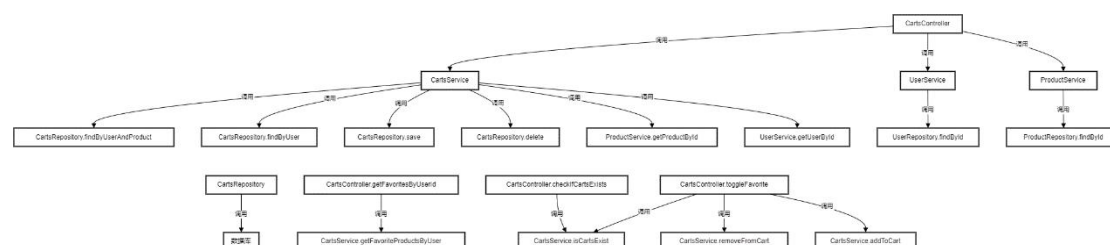
- 返回列表

`ResponseEntity<Boolean>`: 表示商品是否存在于收藏夹中（true 为存在，false 为不存在）。

4.3.1.6 限制条件及出错处理

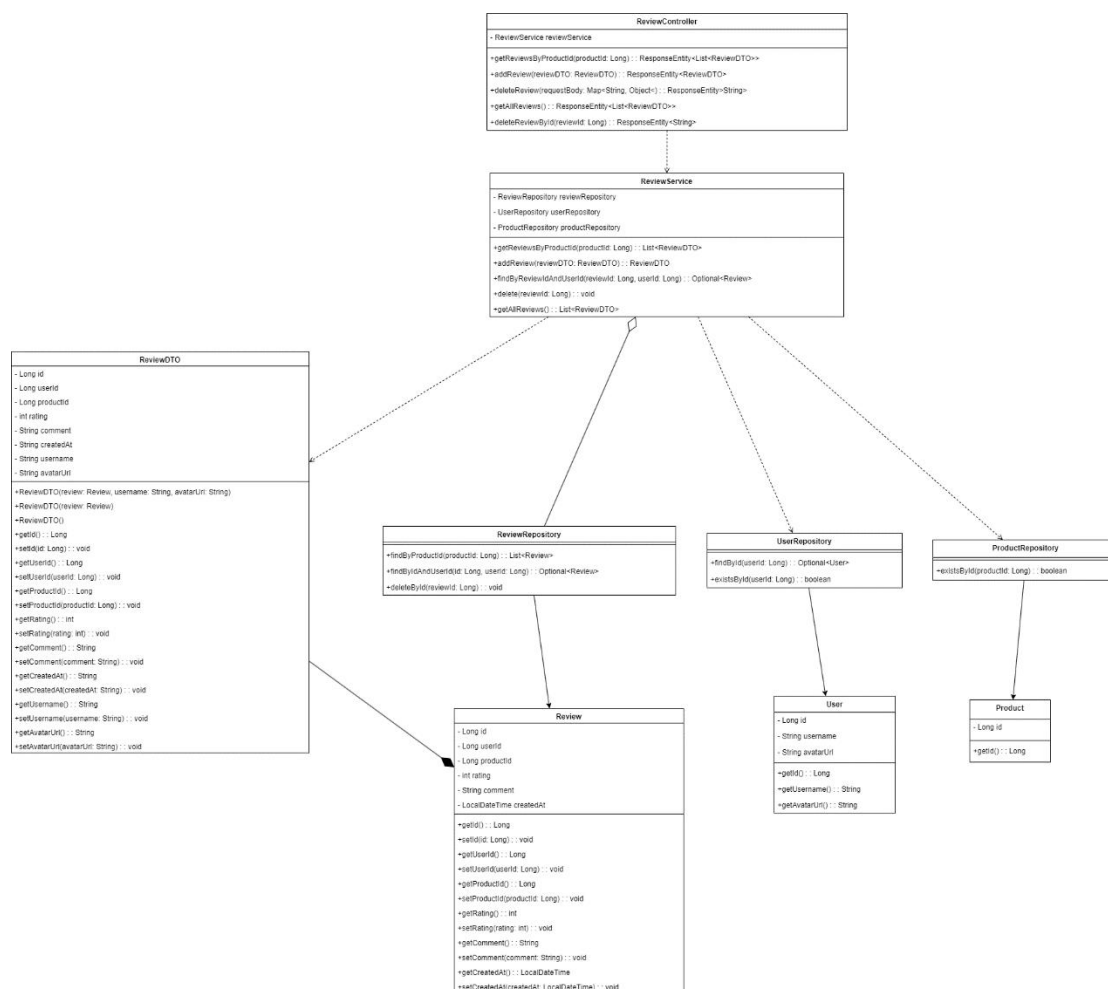
无明显限制条件，只要注意逻辑，用户不能收藏多个相同物品，收藏后，再次点击就是取消收藏。

4.3.2 类调用图



4.4 评论服务

基于 Java Spring Boot 结构，Service 处理业务逻辑；Controller 为接口类，提供方法；Repository 为持久类，用于访问操作数据库。这里的 Review 业务类用于评论服务。



4.4.1 ReviewController 类

4.4.1.1 功能

提供评论管理相关的接口，包括获取商品评论、添加评论、删除评论，以及管理员删除评论。

4.4.1.2 性能

使用 RESTful 风格设计，提供轻量级的 HTTP 请求接口。

通过 ReviewService 实现业务逻辑分离，提高代码复用性和可维护性。

对于评论操作，提供了用户权限校验和管理员权限的区分处理。

4.4.1.3 输入项

商品 ID (productId)：Long 类型，用于查询指定商品的评论。

评论信息 (ReviewDTO)：ReviewDTO 类型，包含评论内容。

用户 ID 和评论 ID (userId、reviewId)：Long 类型，用于定位和操作评论。

4.4.1.4 输出项

评论列表 (List<ReviewDTO>)：返回商品评论或全部评论信息。

操作结果信息：返回成功或失败的消息。

4.4.1.5 方法详细说明

4.4.1.5.1 getReviewsByProductId 方法

- 方法签名

`ResponseEntity<List<ReviewDTO>> getReviewsByProductId(Long productId)`

- 功能说明

根据商品 ID 获取该商品的所有评论。

- 接口调用

调用 `ReviewService` 的 `getReviewsByProductId(Long productId)` 方法。

- 参数列表

`productId`: `Long` 类型，商品 ID。

- 返回列表

`ResponseEntity<List<ReviewDTO>>`: 返回商品的评论列表。

4.4.1.5.2 addReview 方法

- 方法签名

`ResponseEntity<ReviewDTO> addReview(ReviewDTO reviewDTO)`

- 功能说明

添加用户对商品的评论。

- 接口调用

调用 `ReviewService` 的 `addReview(ReviewDTO reviewDTO)` 方法。

- 参数列表

`reviewDTO`: `ReviewDTO` 类型，包含评论内容。

- 返回列表

`ResponseEntity<ReviewDTO>`: 返回保存成功的评论信息。

4.4.1.5.3 deleteReview 方法

- 方法签名

`ResponseEntity<String> deleteReview(Map<String, Object> requestBody)`

- 功能说明

根据用户 ID 和评论 ID 删除用户自己的评论。

- 接口调用

调用 `ReviewService` 的 `findByReviewIdAndUserId(Long reviewId, Long userId)` 和 `delete(Long reviewId)` 方法。

- 参数列表

`requestBody`: `Map<String, Object>` 类型，包含 `userId` 和 `reviewId`。

- ResponseEntity<String>**: 返回删除操作的结果信息。

• 方法签名

• 功能说明

- 接口调用

• 参数列表

无。

- 返回列表

ResponseEntity<List<ReviewDTO>>: 返回所有评论的列表。

• 方法签名

• 功能说明

管理员删除指定 ID 的评论。

- 接口调用

调用 ReviewService 的 delete(Long reviewId) 方法。

- ## • 参数列表

reviewId: Long 类型, 评论 ID。

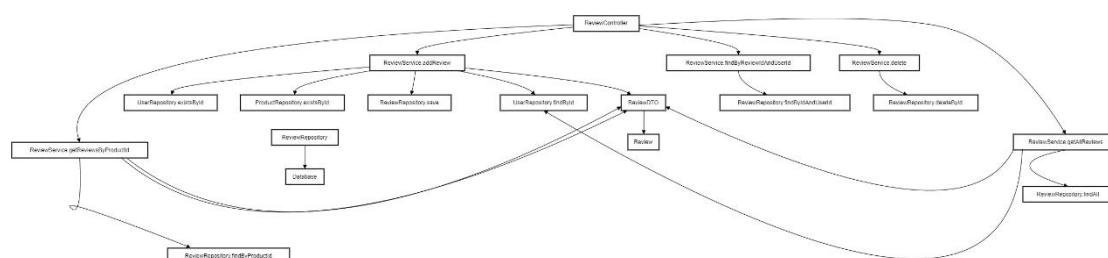
- 返回列表

`ResponseEntity<String>`: 返回删除操作的结果信息。

4.4.1.6 限制条件及出错处理

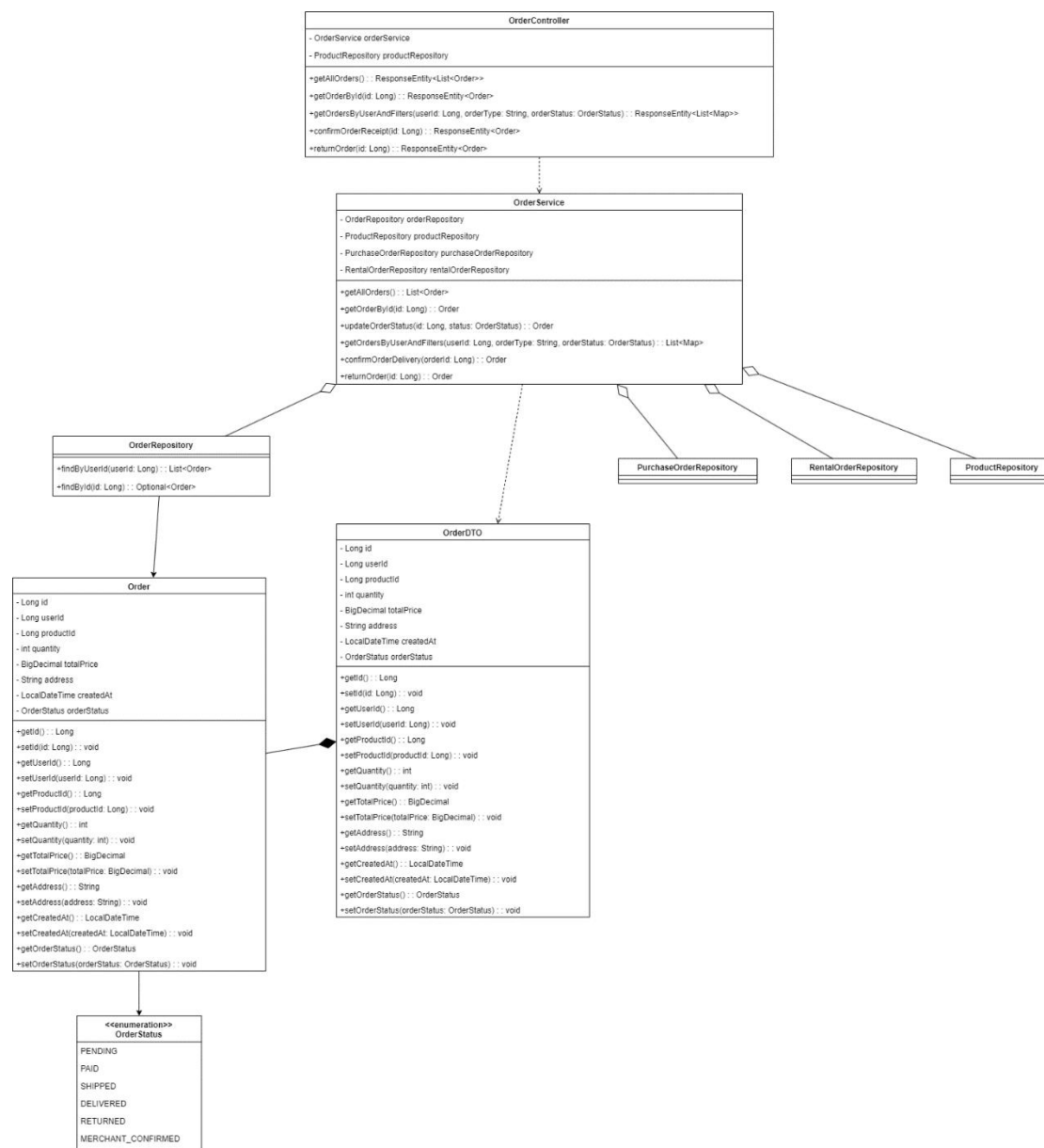
普通用户只能删除自己的评论，而管理员可以删除所有用户的评论。

4.4.2 类调用图



4.5 支付订单管理服务

基于 Java Spring Boot 结构，Service 处理业务逻辑；Controller 为接口类，提供方法；Repository 为持久类，用于访问操作数据库。这里的 Order 业务类用于订单管理服务。



4.5.1 OrderController 类

4.5.1.1 功能

提供订单管理相关的 RESTful 接口，包括查询所有订单、单个订单、按条件筛选订单，以及确认收货和申请还货等操作。

4.5.1.2 性能

使用 RESTful 风格的 HTTP 方法实现轻量化的接口。

通过 OrderService 分离业务逻辑，提升代码的复用性和维护性。

提供筛选订单接口，支持多条件动态查询，优化用户查询体验。

4.5.1.3 输入项

用户 ID (userId) : Long 类型，用于标识用户。

订单 ID (id) : Long 类型，用于标识订单。

订单类型 (orderType) : String 类型，可选值为 "PURCHASE" 或 "RENTAL"。

订单状态 (orderStatus) : OrderStatus 枚举类型，用于标识订单状态。

4.5.1.4 输出项

查询订单时返回 List<Order> 或 Order 类型的订单信息。

操作订单时返回 Order 类型的更新结果。

4.5.1.5 方法详细说明

4.5.1.5.1 getAllOrders 方法

- 方法签名

ResponseEntity<List<Order>> getAllOrders()

- 功能说明

查询系统中的所有订单信息。

- 接口调用

调用 OrderService 的 getAllOrders() 方法。

- 参数列表

无。

- 返回列表

ResponseEntity<List<Order>>: 返回所有订单的列表。

4.5.1.5.2 getOrderById 方法

- 方法签名

ResponseEntity<Order> getOrderById(Long id)

- 功能说明

根据订单 ID 查询订单详情。

- 接口调用

调用 OrderService 的 getOrderById(Long id) 方法。

- 参数列表

id: Long 类型，订单 ID。

- 返回列表

ResponseEntity<Order>: 返回指定订单的详细信息。

4.5.1.5.3 getOrdersByUserAndFilters 方法

- 方法签名

`ResponseEntity<List<Map<String, Object>>>`

`getOrdersByUserAndFilters(Long userId, String orderType, OrderStatus orderStatus)`

- 功能说明

根据用户 ID、订单类型和订单状态筛选订单。

- 接口调用

调用 `OrderService` 的 `getOrdersByUserAndFilters(Long userId, String orderType, OrderStatus orderStatus)` 方法。

- 参数列表

`userId`: `Long` 类型，用户 ID。

`orderType`: `String` 类型，订单类型（可选）。

`orderStatus`: `OrderStatus` 枚举类型，订单状态（可选）。

- 返回列表

`ResponseEntity<List<Map<String, Object>>>`: 返回符合筛选条件的订单列表。

4.5.1.5.4 confirmOrderReceipt 方法

- 方法签名

`ResponseEntity<Order> confirmOrderReceipt(Long id)`

- 功能说明

买家确认收货，更新订单状态。

- 接口调用

调用 `OrderService` 的 `confirmOrderDelivery(Long id)` 方法。

- 参数列表

`id`: `Long` 类型，订单 ID。

- 返回列表

`ResponseEntity<Order>`: 返回更新后的订单信息。

4.5.1.5.5 returnOrder 方法

- 方法签名

`ResponseEntity<Order> returnOrder(Long id)`

- 功能说明

买家申请还货，更新订单状态。

- 接口调用

调用 OrderService 的 returnOrder(Long id) 方法。

- 参数列表

id: Long 类型, 订单 ID。

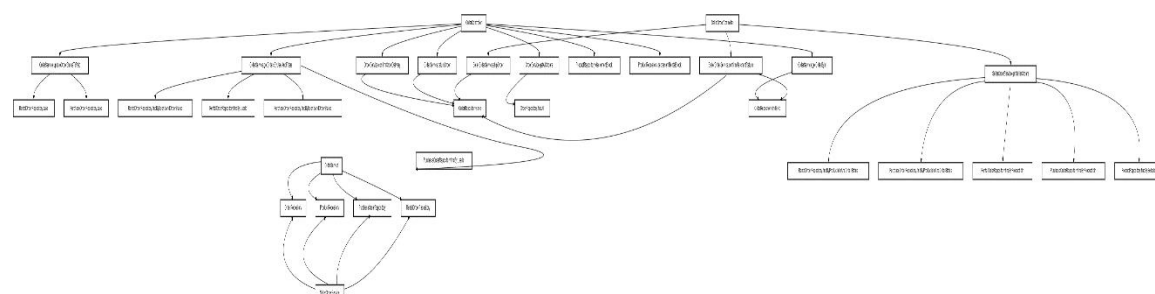
- 返回列表

ResponseEntity<Order>: 返回更新后的订单信息。

4.5.1.6 限制条件及出错处理

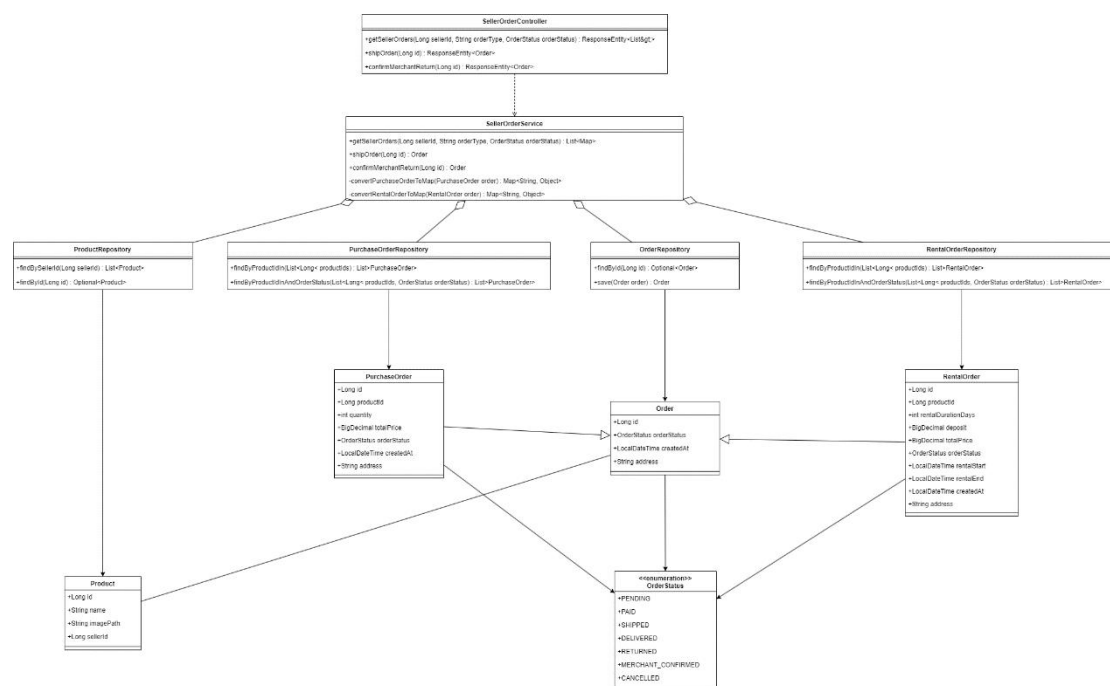
订单状态变化应该由快递公司提供信息, 由于无法对接接口, 我们只能模拟收货等过程。同时, 订单的创建必须有支付流程的限制。

4.5.2 类调用图



4.6 卖出订单管理服务

基于 Java Spring Boot 结构, Service 处理业务逻辑; Controller 为接口类, 提供方法; Repository 为持久类, 用于访问操作数据库。这里的 SellerOrder 业务类用于出售订单管理服务。



4.6.1 SellerOrderController 类

4.6.1.1 功能

提供卖家订单管理的 RESTful 接口，包括查询卖家订单、发货操作和确认退货等功能。

4.6.1.2 性能

使用 RESTful 风格设计，提供轻量化的 HTTP 请求接口。

支持动态筛选订单功能，提高查询灵活性和效率。

分离业务逻辑至 SellerOrderService，提升代码复用性和维护性。

4.6.1.3 输入项

卖家 ID (sellerId)：Long 类型，用于标识卖家。

订单 ID (id)：Long 类型，用于标识订单。

订单类型 (orderType)：String 类型，可选值为 "PURCHASE" 或 "RENTAL"。

订单状态 (orderStatus)：OrderStatus 枚举类型，用于标识订单状态。

4.6.1.4 输出项

查询订单时返回 List<Map<String, Object>> 类型的订单信息。

操作订单时返回 Order 类型的更新结果。

4.6.1.5 方法详细说明

4.6.1.5.1 getSellerOrders 方法

- 方法签名

ResponseEntity<List<Map<String, Object>>> getSellerOrders(Long sellerId, String orderType, OrderStatus orderStatus)

- 功能说明

查询卖家订单，支持按订单类型和订单状态筛选。

- 接口调用

调用 SellerOrderService 的 getSellerOrders(Long sellerId, String orderType, OrderStatus orderStatus) 方法。

- 参数列表

sellerId: Long 类型，卖家 ID。

orderType: String 类型，订单种类（可选）。

orderStatus: OrderStatus 枚举类型，订单状态（可选）。

- 返回列表

ResponseEntity<List<Map<String, Object>>>: 返回符合条件的卖家订单列表。

4.6.1.5.2 shipOrder 方法

• 方法签名

```
ResponseEntity<Order> shipOrder(Long id)
```

• 功能说明

卖家发货，更新订单状态为“已发货”。

• 接口调用

调用 `SellerOrderService` 的 `shipOrder(Long id)` 方法。

• 参数列表

id: Long 类型, 订单 ID。

- 返回列表

ResponseEntity<Order>: 返回更新后的订单信息。

4.6.1.5.3 confirmMerchantReturn 方法

• 方法签名

```
ResponseEntity<Order> confirmMerchantReturn(Long id)
```

• 功能说明

卖家确认退货，更新订单状态为“退货已确认”。

- 接口调用

调用 SellerOrderService 的 confirmMerchantReturn(Long id) 方法。

• 参数列表

id: Long 类型, 订单 ID。

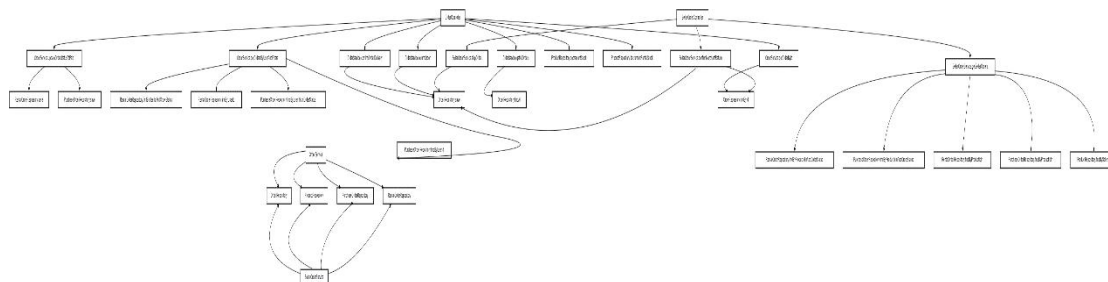
- 返回列表

ResponseEntity<Order>: 返回更新后的订单信息。

4.6.1.6 限制条件及出错处理

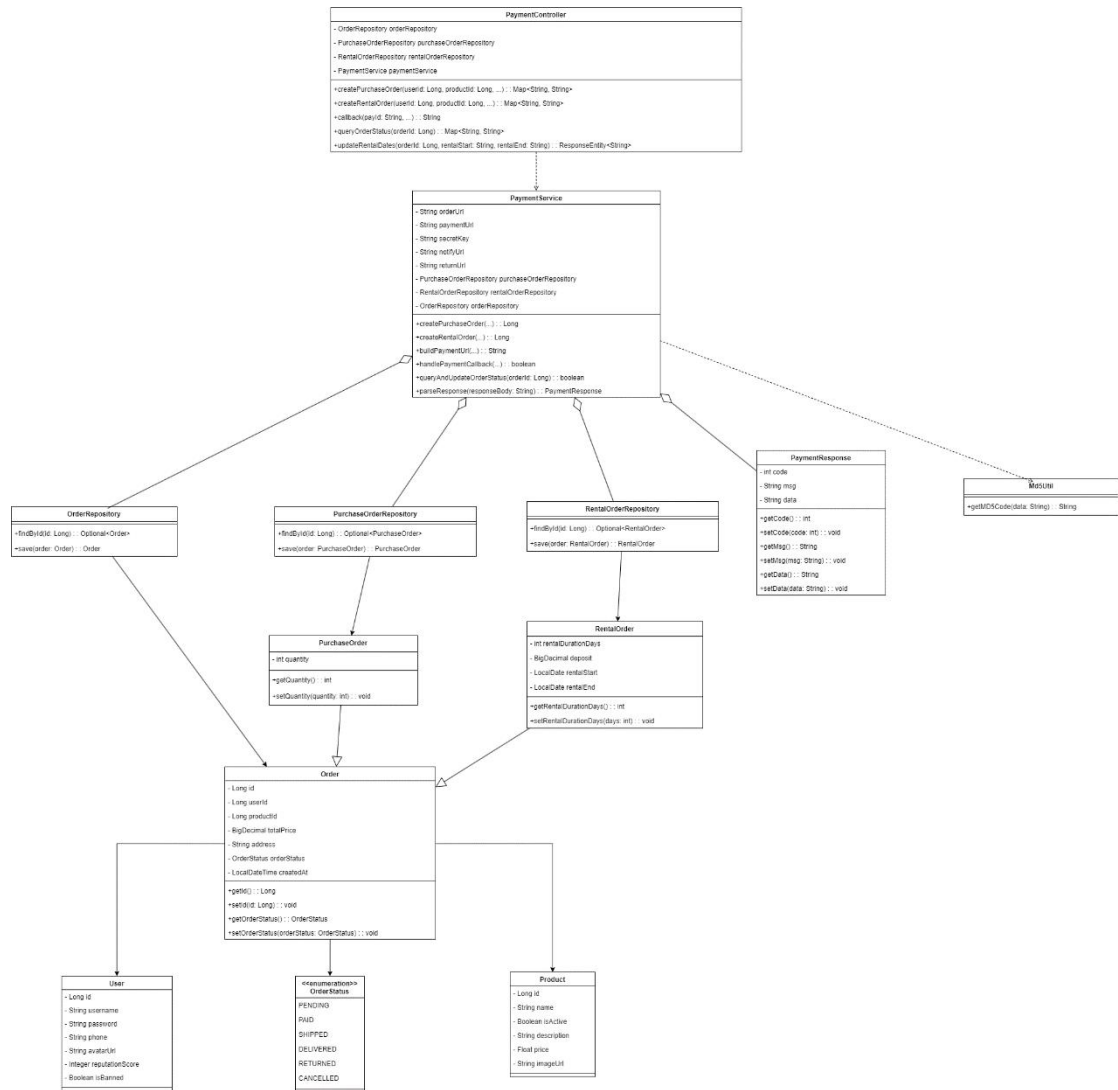
快递公司提供服务，将卖家的订单操作进行状态变更，这里不做讨论。

4.6.2 类调用图



4.7 支付服务

基于 Java Spring Boot 结构，Service 处理业务逻辑；Controller 为接口类，提供方法；Repository 为持久类，用于访问操作数据库。这里的 Payment 业务类用于支付服务。



4.7.1 SellerOrderController 类

4.7.1.1 功能

提供支付管理的 RESTful 接口，包括创建购买和租赁订单、处理支付回调、查询订单状态和更新租赁订单日期等功能。

4.7.1.2 性能

通过 PaymentService 分离业务逻辑，确保接口高效且易于维护。

支持多种支付方式（如微信支付、支付宝支付），增强系统兼容性。

提供订单状态查询和动态更新，提升用户体验。

4.7.1.3 输入项

用户 ID (userId)：Long 类型，用于标识用户。

商品 ID (productId) : Long 类型, 用于标识商品。

支付相关参数 (quantity、price、type、address 等)。

订单回调参数 (payId、param、type 等)。

租赁时间参数 (rentalStart 和 rentalEnd)。

4.7.1.4 输出项

创建订单时返回 Map<String, String> 类型的订单信息和支付跳转链接。

查询订单状态或更新操作返回 Map<String, String> 或

ResponseEntity<String> 类型的结果。

4.7.1.5 方法详细说明

4.7.1.5.1 createPurchaseOrder 方法

- 方法签名

Map<String, String> createPurchaseOrder(Long userId, Long productId, Integer quantity, int type, BigDecimal price, String address)

- 功能说明

创建购买订单并生成支付跳转链接。

- 接口调用

调用 PaymentService 的 createPurchaseOrder 和 buildPaymentUrl 方法。

- 参数列表

userId: Long 类型, 用户 ID。

productId: Long 类型, 商品 ID。

quantity: Integer 类型, 购买数量。

type: int 类型, 支付方式 (微信/支付宝)。

price: BigDecimal 类型, 订单价格。

address: String 类型, 收货地址。

- 返回列表

Map<String, String>: 订单信息和支付跳转链接。

4.7.1.5.2 createRentalOrder 方法

- 方法签名

Map<String, String> createRentalOrder(Long userId, Long productId, int days, BigDecimal deposit, BigDecimal price, Integer quantity, String address, Model model)

- 功能说明

创建租赁订单并生成支付跳转链接。

- 接口调用

调用 PaymentService 的 createRentalOrder 和 buildPaymentUrl 方法。

- 参数列表

同 createPurchaseOrder 方法，增加租赁天数 days 和押金 deposit。

- 返回列表

Map<String, String>: 订单信息和支付跳转链接。

4.7.1.5.3 callback 方法

- 方法签名

String callback(String payId, String param, int type, double price, double reallyPrice, String sign)

- 功能说明

接收支付平台的回调信息并处理订单状态。

- 接口调用

调用 PaymentService 的 handlePaymentCallback 方法。

- 参数列表

回调参数: payId、param、type、price、reallyPrice、sign。

- 返回列表

String: 支付处理结果 (success 或 fail) 。

4.7.1.5.4 queryOrderStatus 方法

- 方法签名

Map<String, String> queryOrderStatus(Long orderId)

- 功能说明

查询订单状态并更新订单信息。

- 接口调用

调用 PaymentService 的 queryAndUpdateOrderStatus 方法。

- 参数列表

orderId: Long 类型，订单 ID。

- 返回列表

Map<String, String>: 订单状态信息。

4.7.1.5.5 updateRentalDates 方法

- 方法签名

ResponseEntity<String> updateRentalDates(Long orderId, String rentalStart, String rentalEnd)

- 功能说明

更新租赁订单的租借起始和结束日期。

- 接口调用

调用 `RentalOrderRepository` 的 `findById` 和 `save` 方法。

• 参数列表

orderId: Long 类型, 订单 ID。

rentalStart 和 rentalEnd: String 类型，租赁时间范围。

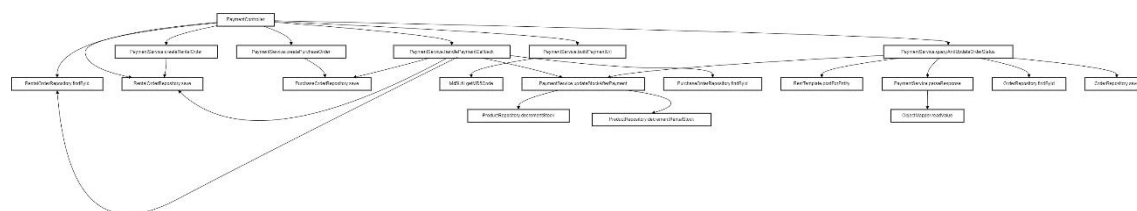
- 返回列表

ResponseEntity<String>: 更新结果信息。

4.7.1.6 限制条件及出错处理

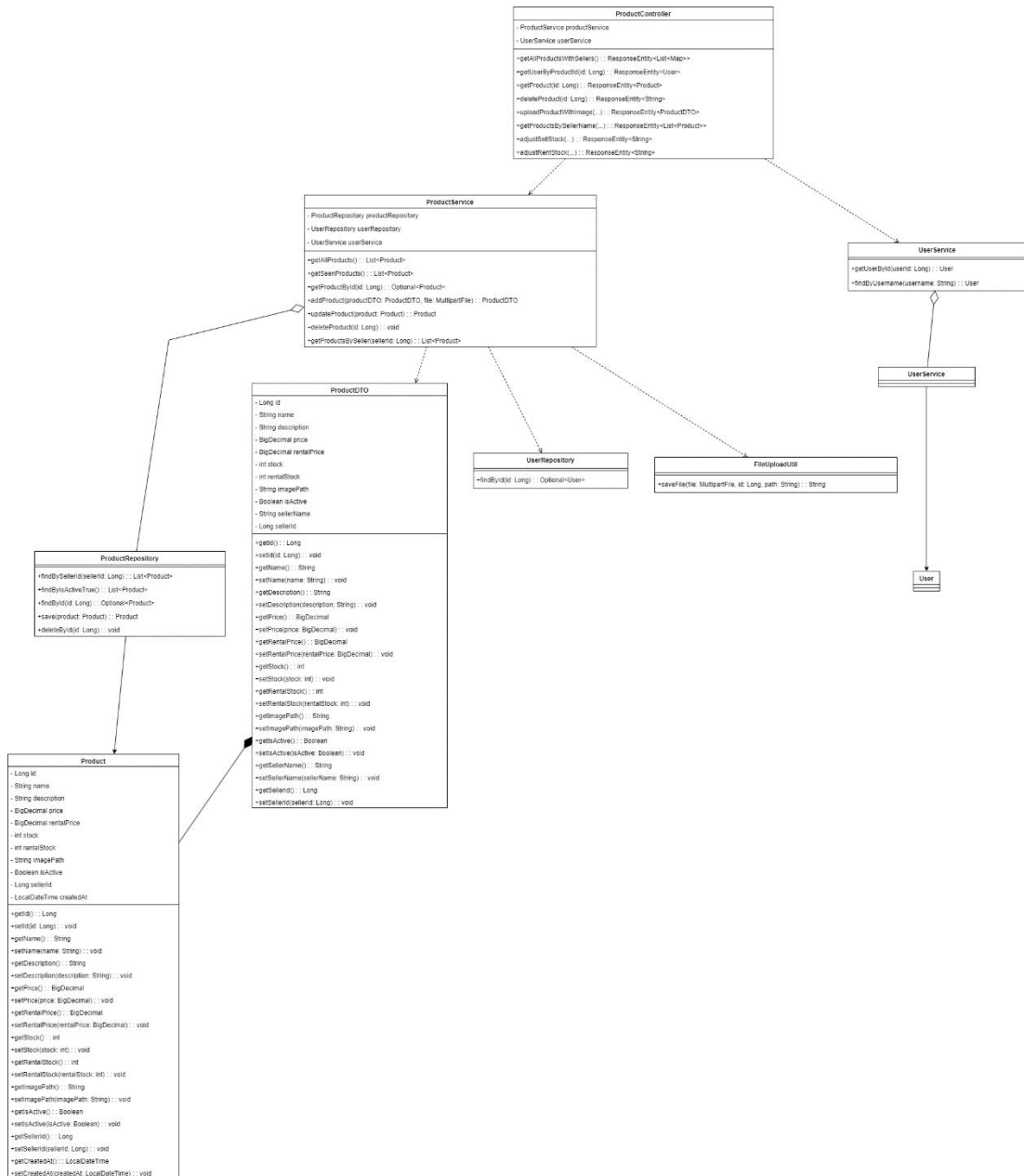
订单已支付必须确保金钱已经转账后才能进行。

4.7.2 类调用图



4.8 商品管理服务

基于 Java Spring Boot 结构，Service 处理业务逻辑；Controller 为接口类，提供方法；Repository 为持久类，用于访问操作数据库。这里的 Product 业务类用于产品管理服务。



4.8.1 ProductController 类

4.8.1.1 功能

提供商品管理的 RESTful 接口，包括获取商品信息、添加商品、调整库存、查询卖家商品等功能。

4.8.1.2 性能

使用 ProductService 和 UserService 分离业务逻辑，保证代码模块化和可维护性。

提供基于条件查询商品和卖家信息的接口，提升查询效率和灵活性。

支持文件上传操作，用于商品图片管理。

4.8.1.3 输入项

商品信息：ProductDTO 对象或相关字段（如名称、价格、库存等）。

商品 ID (productId)：Long 类型，用于标识商品。

卖家信息：sellerId 或 sellerName。

调整库存参数：quantity 表示调整值。

文件上传：MultipartFile 类型，用于上传商品图片。

4.8.1.4 输出项

商品列表或单个商品的详细信息。

操作成功或失败的结果消息。

4.8.1.5 方法详细说明

4.8.1.5.1 getAllProductsWithSellers 方法

- 方法签名

`ResponseEntity<List<Map<String, Object>>> getAllProductsWithSellers()`

- 功能说明

获取所有商品及对应卖家信息。

- 接口调用

调用 `ProductService.getSeenProducts` 和 `UserService.getUserById` 方法。

- 参数列表

无。

- 返回列表

`ResponseEntity<List<Map<String, Object>>>`：包含商品和卖家信息的列表。

4.8.1.5.2 getUserByProductId 方法

- 方法签名

`ResponseEntity<User> getUserByProductId(Long id)`

- 功能说明

根据商品 ID 获取对应的卖家信息。

- 接口调用

调用 `ProductService.getProductById` 和 `UserService.getUserById` 方法。

- 参数列表

id: Long 类型，商品 ID。

- 返回列表

`ResponseEntity<User>`：返回卖家信息。

4.8.1.5.3 deleteProduct 方法

- 方法签名

`ResponseEntity<String> deleteProduct(Long id)`

- **功能说明**

下架商品，更新商品状态为 `isActive = false`。

- **接口调用**

调用 `ProductService.getProductById` 和 `ProductService.updateProduct` 方法。

- **参数列表**

`id`: `Long` 类型，商品 ID。

- **返回列表**

`ResponseEntity<String>`: 返回下架结果消息。

4.8.1.5.4 `uploadProductWithImage` 方法

- **方法签名**

`ResponseEntity<ProductDTO> uploadProductWithImage(HttpServletRequest request, ...)`

- **功能说明**

上传商品信息及商品图片。

- **接口调用**

调用 `ProductService.addProduct(ProductDTO, MultipartFile)` 方法。

- **参数列表**

包含商品信息的参数（如 `name`、`price`、`sellerId` 等）。

`file`: `MultipartFile` 类型，商品图片文件。

- **返回列表**

`ResponseEntity<ProductDTO>`: 返回保存成功的商品信息。

4.8.1.5.5 `getProductsBySellerName` 方法

- **方法签名**

`ResponseEntity<List<Product>> getProductsBySellerName(Map<String, String> request)`

- **功能说明**

根据卖家用户名查询其上架商品。

- **接口调用**

调用 `UserService.findByUsername` 和 `ProductService.getProductsBySeller` 方法。

- **参数列表**

`request`: 包含卖家用户名的请求体。

- 返回列表

ResponseEntity<List<Product>>: 返回卖家的商品列表。

4.8.1.5.6 adjustSellStock 方法

- 方法签名

ResponseEntity<String> adjustSellStock(Map<String, Object> request)

- 功能说明

调整商品的出售库存。

- 接口调用

调用 ProductService.getProductById 和 ProductService.updateProduct 方法。

- 参数列表

request: 包含 productId 和 quantity 的请求体。

- 返回列表

ResponseEntity<String>: 返回库存调整结果消息。

4.8.1.5.7 adjustRentStock 方法

- 方法签名

ResponseEntity<String> adjustRentStock(Map<String, Object> request)

- 功能说明

调整商品的租赁库存。

- 接口调用

调用 ProductService.getProductById 和 ProductService.updateProduct 方法。

- 参数列表

request: 包含 productId 和 quantity 的请求体。

- 返回列表

ResponseEntity<String>: 返回库存调整结果消息。

4.8.1.6 限制条件及出错处理

对卖家上传商品图片大小进行限制，管理员进行不良商品的下架。

4.8.2 类调用图

